COMP426 - Assignment 2

Patrick Deniso (40194944)

I implemented Conway's Game of Life (optionally multi-species) in C++20 using oneTBB. The CPU does all the simulation; OpenGL is used only to display pixels at ~30 FPS. The program used two roles:

- **Control thread (main):** owns timing and rendering.
- **Computation threads (workers):** update the grid in parallel.

Now, the TBB version preserves the same control/computation separation, but replaces manual threads + barrier with TBB's task scheduler and `parallel_for`.

Mapping A1 -> TBB

A1 (manual threads using std::barrier)

- N worker threads ran step_rows(y0, y1) on disjoint row blocks.
- All workers hit barrier.arrive_and_wait().
- Main swapped next <-> curr, built the RGBA buffer, uploaded, and drew.

A2 (TBB)

- The main thread launches a parallel_for each frame to compute next from curr.
- parallel_for waits before returning (implicit sync), so no explicit barrier is needed.
- After it returns, main swaps buffers, builds RGBA, uploads, draws, and paces to ~30 FPS (same as A1).

Data Layout (unchanged from A1)

- Grid size: 1024×768, row-major indexing `i = y*W + x`.
- Double buffering per species: `layers_curr[s]` (read-only this frame) and `layers_next[s]` (write-only).
- This avoids locks: all threads read the same “old” state and write to separate memory.

Optimizations

- `parallel_for` with a reasonable **grain size**
- Single swap location on main after compute
- Avoiding locks in the hot path
- `steady_clock` + `sleep_until` pacing for a consistent frame time

Build & Run

This is for macOS with Homebrew.

```
brew install tbb glfw
cmake -S . -B build \
  -Dglfw3_DIR=/opt/homebrew/opt/glfw/lib/cmake/glfw3 \
  -DTBB_DIR=/opt/homebrew/opt/tbb/lib/cmake/TBB
cmake --build build -j
./build/game [species]
```

Complexity

Per frame: **O(S * W * H)** cell updates, scaling well across CPU cores due to no contention.